# Group 11 Deliverable 1B

AIDAN ROBERTSON, JACK CONNOLLY, DANIEL JORNEA, MERIC KARACAN, CASPAR LEWIS, JOHN OSBORNE, VIKTOR POPOV, and KOUROSH SAEIDI

## 1 Requirements Engineering and Analysis

### 1.1 Project Overview

Notebuddy is a note-sharing platform designed for students at the University of Bath. We have found students lack a centralized system for sharing notes and ensuring quality revision time. Several issues were identified while communicating with stakeholders: Students waste time searching for notes in fragmented sources (e.g. WhatsApp, Discord, Instagram). These channels offer no guarantee on the quality of notes. Also there's no way for lecturers to mark reliable, high-quality notes, or correct any misunderstandings in circulated notes. As well as this lack of collaboration limits the cohort in effectively improving their course knowledge.

Our solution aims to allow students to share, edit, rate and provide suggestions to notes. Through the use of lecturer verified tags and community suggestions we aim to increase the note quality standard on the platform. Gamification elements could be added to motivate users to use the platform, share notes and collaborate, to ensure long-term use of the system. Overall this platform should allow for students to ensure the quality of the notes they use and enhance a sense of community towards their studies. As for the lecturers, they will have increased assurance that students are using relevant study materials.

### 1.2 Requirements Establishment

Interviews with students and lecturers and questionnaires were used to establish primary requirements and check what to prioritise [Appendix A.1]. Similar solutions such as Studocu [1] and Studydrive [2] were assessed and used to inform the project.

We began by gathering stakeholder data (3 interviews and 26 survey responses) which were used to inform the following requirements: Reliability rating for uploaded notes (84.6% of respondents) a Must Have feature to ensure note quality. Note suggestions (57.7% of respondents) a Should Have feature to ensure note quality and collaboration Downloading Notes (65.4% of respondents) a Must Have feature to allow notes to be available offline. Public account only (88.5% of respondents reported that they would trust notes more from public accounts rather than private ones) a Must Have feature

Studocu is a similar solution to ours – you can access notes by entering a university name and searching by course/module. Furthermore, it allows you to access the content without making an account, and surprisingly a lot of the material are uploads of lecture content rather than revision material/notes. The platform also doesn't allow you to search by course and only by module, which is inconvenient when trying to find a specific material – furthermore it displays information irrelevant to the material. It also displays past modules that aren't available anymore. Notebuddy will try to mitigate these issues by only allowing students to access content of their course, and deleting any unavailable modules from the database. Furthermore its rating system for notes was a simple good/bad rating from users, which

Authors' Contact Information: Aidan Robertson, ar3070@bath.ac.uk; Jack Connolly, jc4110@bath.ac.uk; Daniel Jornea, dj660@bath.ac.uk; Meric Karacan, mk2946@bath.ac.uk; Caspar Lewis, cl2732@bath.ac.uk; John Osborne, jwo27@bath.ac.uk; Viktor Popov, vip28@bath.ac.uk; Kourosh Saeidi, ks2805@bath.ac.uk.

may not allow users to give enough detail about how they felt the notes were, we will address this by allowing for further commenting and a 5 star rating system. Studocu enforces users to upload their notes in order to continue to use the website, which could be a good incentive for users to upload notes but could also lead to a reduced quality of uploaded notes as users just upload whatever they can in order to keep reading. We will not implement this in our project, as the quality of the notes are paramount. Instead we will offer students benefits through gamification to encourage them to upload notes.

Similarly Studydrive, also allows you to search for university modules without making an account. Also Studydrive allows you to see when notes have been uploaded, a useful feature allowing users to check if they're reading up-to-date notes. Notebuddy will provide different filtering mechanisms for the review, allowing students to check if they're still relevant. Studydrive allows users to leave comments on uploaded notes which is a feature that should be included in our platform in order to allow users to request clarification. Unfortunately Studydrive provides poor formatting for mathematical symbols and equations. Notebuddy will try to provide an intuitive and simple system to format mathematical equations using LaTeX API.

### 1.3 System Environment

Notebuddy will mostly be used by students and some lecturers from the University of Bath, the students will mostly want Notebuddy to be a platform that allows them to easily share, read and edit their notes in collaboration with other students in order to improve their understanding of the content. Lecturers will also want Notebuddy to improve the understanding that their students have of the content, while not compromising the students ability to learn or allowing their lectures to be shared outside the organisation.

The students will want to use Notebuddy to share, read and edit notes. A concern is that students might refuse to upload the notes that they have taken time to make in order to help others. However, sharing personal notes can help improve the uploader's understanding as other users can point out any issue they have found in the uploaded notes. Students may be worried about academic integrity of uploading notes, however these uploaded notes should not be considered a complete replacement of note taking and should be used to reinforce understanding.

The lecturers will want to know that the students using Notebuddy are using reliable notes to prevent users from propagating their misunderstandings onto other students. To do this they could upload a set of lecturer verified notes or verify notes that have already been uploaded. The lecturer does not need to do this and can still benefit from Notebuddy without doing so, as the community features of Notebuddy will allow for students to increase the quality of their notes independent of them. Lecturers may also be worried about content they have created being shared outside of the university, however this site would only be accessible by students at the University of Bath, so the risk of students sharing them outside of the organisation would not be increased by Notebuddy.

### 1.4 SMART User Stories and Acceptance Tests

SMART 1): "As a student I want to create notes so I can share my knowledge with others"
SMART 2): "As a lecturer I want to be able to verify notes, to ensure that misunderstandings don't circulate around the platform"
Acceptance tests for story 1:

"GIVEN a student is logged to the platform AND is on the home page WHEN they click on the [Create Note] button THEN a new blank note opens, AND the note is saved as a draft" "GIVEN a student has modified a personal note WHEN they attempt to publish the note THEN a confirmation appears with a module selection dropdown" "GIVEN a student

has opened a blank note WHEN they look at the toolbar THEN they can see different formatting options (Headings, Fonts, Sizes, Bold, Italic, etc.) AND they can see [Save], [Exit], [Publish] buttons"

Acceptance tests for story 2:

"GIVEN that a lecturer has logged in to the platform, AND they are searching for notes to verify WHEN they click on a student note THEN a note opens in read-only mode AND they can see a [Verify Note] button in the toolbar AND they can see current note discussions and ratings"

"GIVEN that a lecturer is viewing a note, WHEN they click on the [Verify Note] button, THEN a confirmation pops up confirming the evaluation AND a Verified badge appears on the note AND the note appears higher in the search list "

"GIVEN that a lecturer has opened a page with the Verified tag which has been modified WHEN they click on [Remove Verification] button THEN the verification badge is removed"

### 1.5 Use Cases

See Tables 1 and 2.

Table 1. Use Case 1

| | |
|---|---|
| **Name** | Create new Note |
| **Actor** | Student |
| **Goal** | Produce a new note for a specific module |
| **Preconditions** | User is logged in with University of Bath account, User is on the dashboard/home section |
| **Basic Flow** | The user selects the Create Note button, The system displays a blank note editor, The user enters text and uses formatting tools, The user selects the course and module for the note, The user clicks Save, The system stores the note and displays it in the user's dashboard |
| **Extension Points** | {Select Module} - when module is chosen, {Save Note} - before save |
| **Alternative Flows** | A1: Missing module information At Select Module, if the user attempts to save without selecting course/module, The system displays an error message. The system returns to the note editor to allow selection. A2: Loss of connection At any point before Save Note The system shows a warning banner indicating the loss of connection. Autosave is paused. |
| **Postconditions** | A new note is created, stored, and visible in the dashboard. |
| **Scenarios** | Carlos wants to create a new note about Machine Learning and share it publicly. Flow: Carlos logs in to Notebuddy, and is on the home page. He clicks the Create Note button, which opens up a new blank note. Carlos writes up his understanding of the content on the page He changes the title Carlos clicks on the Publish Note button to make the note public. Other people can now view the note, suggest edits and rate the quality of Carlos' note. |

Table 2. Use Case 2

| | |
|---|---|
| **Name** | Verify student note |
| **Actor** | Lecturer |
| **Goal** | Mark a note as accurate and aligned with module content. |
| **Preconditions** | Lecturer is logged in. Lecturer has opened a note. |
| **Basic Flow** | The lecturer opens a note. The system displays the Verify Note option. The lecturer reviews the content. The lecturer clicks the Verify tag. The system flags the note as verified and adds timestamp and lecturer ID. |
| **Extension Points** | {Verify Tag} - when the lecturer clicks on the Verify tag |
| **Alternative Flows** | B1: Lecturer removes verification Triggered at Verify Tag The lecturer clicks Remove Verification. The system removes the tag and updates the metadata. B2: Verification removed due to outdated note At Verify Tag If the note is older than a threshold (e.g, >5 years) The system warns the lecturer about potential outdated material. Lecturer may override or cancel verification. |
| **Postconditions** | The note becomes marked as verified or remains unverified if cancelled. |
| **Scenarios** | Later in the evening Dr. Rhymes wants to check the quality of notes on Notebuddy: Rhymes logs into Notebuddy He navigates to the notes section He sees [Overview of OOP Languages] note He opens the note, and reviews the quality. Dr. Rhymes clicks on the [Verify Note] button. A confirmation message appears, saying that the note has been verified. And the Verified badge is displayed. |

### 1.6 Core Requirements List

We have identified functional and non-functional requirements for the project and group them into six logical groups as described below, using MoScoW prioritisation (Must Have, Should Have, Could Have and Won't Have). Account Management (AM) Note Management (NM) Gamification (GM) Security (SEC) Accessibility (ACC) System Performance (P)

Table 3. Account Management

| ID | Name | Type | Priority | Description |
|---|---|---|---|---|
| AM1 | Account creation | Functional | M | Sign up using University of Bath email. |
| AM2 | Account deletion | Functional | M | Users can delete their account from the settings page. |
| AM3 | Account grouping | Functional | M | User accounts are automatically grouped by course. |
| AM4 | Password recovery | Functional | W | Not needed, since authentication is handled through the University of Bath account system. |
| AM5 | 2-Factor Auth | Functional | W | Provides an optional additional security layer. |
| AM6 | Email verification | Functional | W | Not required because accounts are linked to the university identity system. |
| AM7 | Privileges | Functional | M | Students: create notes, suggest edits, rate notes.<br>Lecturers: all student privileges + add Verify Tag. |
| AM8 | Profile management | Functional | C | Custom names, descriptions, and profile pictures. |

Table 4. Note Management

| ID | Name | Type | Priority | Description |
|---|---|---|---|---|
| NM1 | Create Notes | Functional | M | Students and lecturers can create notes. |
| NM2 | Delete Notes | Functional | M | Users can delete notes. |
| NM3 | Edit Notes | Functional | M | Only the note owner can modify their own notes. |
| NM4 | Suggest Edits | Functional | M | All users can make suggestions on any note. |
| NM5 | Verify Tag | Functional | M | Lecturers can verify notes. |
| NM6 | Folder Structure | Functional | M | Notes are grouped by course and module. |
| NM7 | Pin Notes | Functional | S | Users can pin favourite notes to their dashboard. |
| NM8 | Auto-save | Non-functional | S | Changes are automatically saved every 10 seconds. |
| NM9 | Version-control | Functional | C | Changes are saved as time-stamped versions. |
| NM10 | Note Dashboard | Functional | S | Dashboard on the home page showing recently visited notes, pinned notes, and personal notes. |
| NM11 | Filtering | Functional | M | Users can filter notes by Module, Date, Verified status, Popularity, and Reviews. |
| NM12 | Searching | Functional | M | Users can search notes by name. |
| NM13 | Editing Tools | Functional | M | Includes essential editing tools: font changes, headers, font size, bold, etc. |
| NM14 | Mathematical Equations | Functional | S | Support for formatting mathematical equations. |
| NM15 | Code Blocks | Functional | C | Users can include code blocks, e.g., through markdown syntax. |
| NM16 | Note Rating | Functional | S | Users can rate notes from 1 to 5 stars. |
| NM17 | Note Viewing | Functional | M | Users can view notes. |

Table 5. Gamification

| ID | Name | Type | Priority | Description |
|---|---|---|---|---|
| GAM1 | Leaderboard | Functional | C | Graph displays user rankings. |
| GAM2 | Module Ranks | Functional | C | Leaderboards organised by module. |
| GAM3 | Weekly Ranks | Functional | C | Weekly rankings updated every week. |
| GAM4 | Monthly Ranks | Functional | C | Rankings updated monthly. |
| GAM5 | All-time Ranks | Functional | C | All-time contributions grouped by module and aggregated. |
| GAM6 | Badges | Functional | C | Awards given for reaching certain milestones. |
| GAM7 | Levels | Functional | C | Experience (EXP) awarded for collaborating on the website. |

Table 6. Accessibility

| ID | Name | Type | Priority | Description |
|---|---|---|---|---|
| ACC1 | Colour Contrast | Functional | C | Provide sufficient colour contrast for users with visual impairments. |
| ACC2 | Screen Reader | Functional | C | Support for screen readers to ensure content can be read aloud. |
| ACC3 | Dark/White Mode | Functional | S | Offer light and dark mode options depending on user preference. |
| ACC4 | Mobile Access | Non-functional | C | Ensure the system supports various mobile screen sizes. |
| ACC5 | Export Notes | Non-functional | S | Allow users to export notes for offline use. |

Table 7. Performance

| ID | Name | Type | Priority | Description |
|---|---|---|---|---|
| P1 | Scalability | Non-functional | S | System must handle up to 5000 concurrent users. |
| P2 | Search Speed | Non-functional | S | Retrieve and filter notes in under 1 second. |
| P3 | Page Load | Non-functional | M | Page load time must be under 2 seconds. Any note edits must complete in under 300 ms. |
| P4 | Ranking Update | Non-functional | S | All leaderboards must update every 5 minutes. |
| P5 | Gamification Responses | Non-functional | S | Level-ups, EXP awards, and badges must appear within 1 second. |
| P6 | Note Deletion | Non-functional | S | Deleted notes are moved to trash and permanently deleted after 30 days. |
| P7 | Deleted Notes Recovery | Non-functional | C | Recovered notes must be visible within 3 seconds with no data loss. |
| P8 | Testing | Non-functional | M | Test coverage above 70% is considered satisfactory. |

Table 8. Security

| ID | Name | Type | Priority | Description |
|---|---|---|---|---|
| SEC1 | Login | Non-functional | M | Users must sign in using University of Bath credentials. |
| SEC2 | Password Encryption | Non-functional | M | Passwords must be securely hashed. |
| SEC3 | HTTPS Transmission | Non-functional | M | All data must be encrypted and transmitted using HTTPS protocol. |
| SEC4 | Brute Force Protection | Non-functional | S | Limit login attempts to 3 with a cooldown period to prevent brute-force attacks. |
| SEC5 | GDPR Data Protection | Non-functional | M | Account deletion requests must be completed within 30 days to comply with GDPR. |
| SEC6 | Malware Scanning | Non-functional | S | Content uploaded must be scanned for malware. |
| SEC7 | SQL Injection | Non-functional | M | Use parameterized queries to prevent SQL injection attacks. |

### 1.7 Requirements Conflicts

Whilst researching requirements for the website several conflicts emerged between the desires of different stakeholders. One of the most significant conflicts was between student privacy and system accountability. Survey results showed that students trust notes more when they come from identifiable contributors (88.5%), yet many students also expressed concern about publicly displaying their names. We decided it would be best for student names to be visible as we are aiming to develop a transparent collaborative website.

Conflicts also arose between collaborative editing and content ownership. Students want the ability to edit and improve each other's notes (57.7% requested this), but they may feel protective of their own work. To resolve this, we decided to implement direct edits only to the creator and suggested edits for all viewers. This allows the owner to keep control of their work whilst allowing the rest of the community to contribute allowing collaboration.

We also found that Lecturers' needs also conflicted with students' desires. Lecturers want accurate notes that reflect course content, but they don't want to be burdened with additional work. On the other hand students rely on lecturer approval to be confident in revision material. To balance this, we thought that in addition to lecturer verification we should also assure quality through peer ratings and edit suggestions.

Finally, we observed differences regarding the scope of access. Students want an easy to use, shareable platform, but lecturers worry about their materials leaving the university environment. We resolved this using mandatory University of Bath authentication to ensure that materials are protected.

## 2 Architecture Design

### 2.1 High Level Architecture

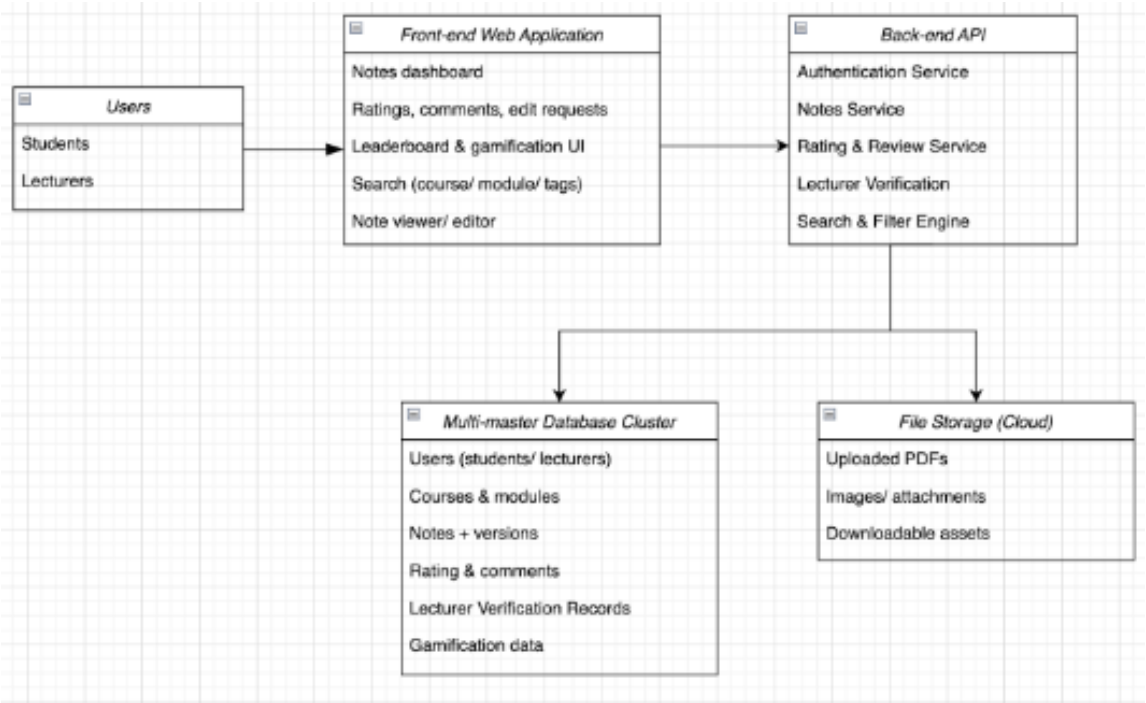


**Figure 1: High Level Architecture Diagram**

Our system allows students and lecturers to access NoteBuddy through the web application, where they can view notes, leave ratings, suggest edits, and navigate content using a hierarchical notes dashboard or a search function. Users can also view their leaderboard position, badges, and levels, which are stored with their account data. To support these features, the backend will expose an API that retrieves notes, ratings, verification status, and search results, as well as validates user emails during account creation.

The system's data will be stored in a multi-master replicated database, where each cluster remains nearly identical. This database will hold user login details, emails, and all gamification data, as well as comments and suggested edits on notes and whether a note is lecturer verified. Notes will also be linked to their associated courses and modules to support sorting and navigation. Instead of storing the full note files directly, the database will store references to their locations in file storage, along with links to previous versions so users can revert to earlier revisions if needed.

## 2.2 Architectural Views

### *2.2.1 Logical.*

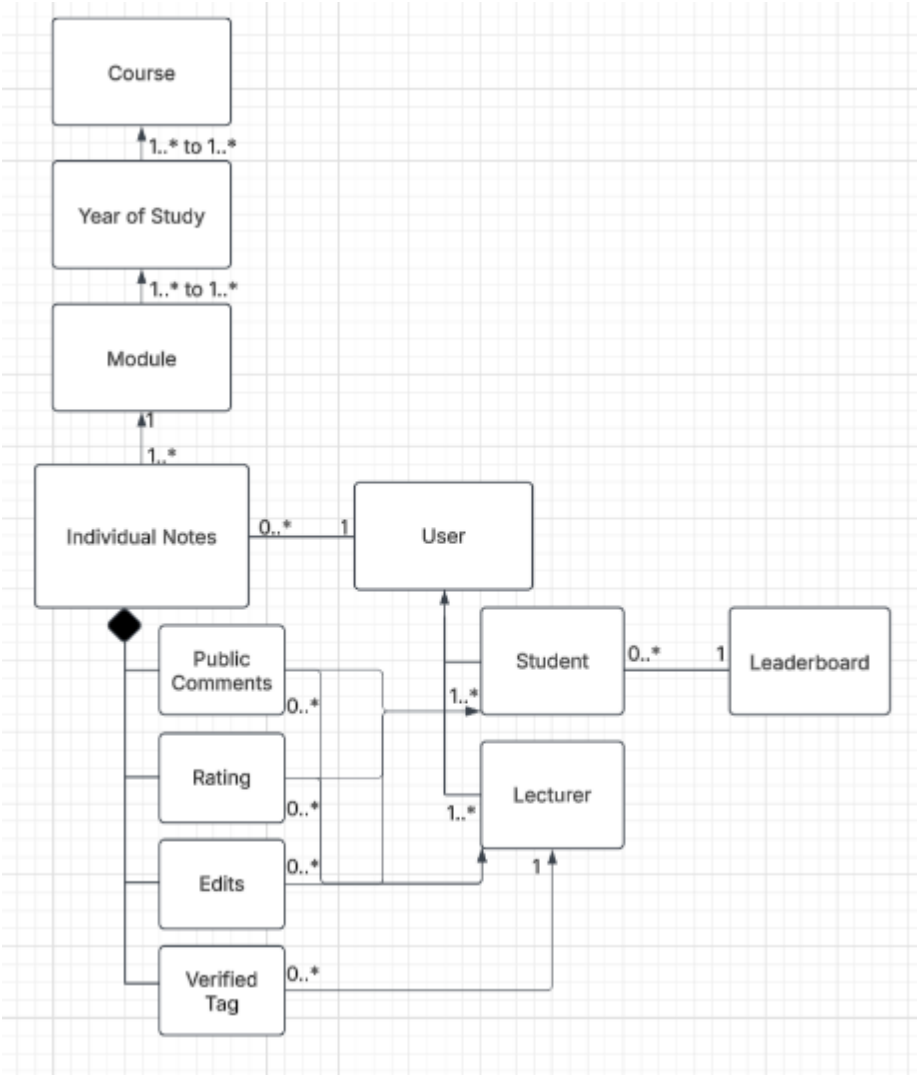


**Figure 2: Logical Architecture Diagram**

Table 9. Logical Element Catalogue

| **Element** | **Purpose** | **Owner** |
|---|---|---|
| Course | Represents the overarching course which a set of notes belong to. | John |
| Year of Study | The year of study which the module belongs to. | John |
| Module | The module name/code of which the notes belong to. | John |
| Individual Notes | The core functionality of the app: a single set of notes for a specific topic, area, or module. | Daniel |
| User | User entity which requires a University of Bath email. | Meric |
| Student | Type of user currently studying at the University of Bath. The only form of user with access to the leaderboard. | Meric |
| Lecturer | Type of user currently teaching at the University of Bath. Only form of user with access to the verified tag. | Meric |
| Leaderboard | A leaderboard for Student users, showing which users have the best notes chosen through peer reviews. | Kourosh |
| Public Comments | Public comments underneath a single note, which can be made by students or lecturers. | Daniel |
| Rating | Rating (0-5) for single notes; can be done by students and lecturers. | Aidan |
| Edits | Suggested and previous edits which can be made by students and lecturers. | Daniel |
| Verified Tag | Indicates that notes have been verified as correct and useful. Can only be accessed by lecturers. | Victor |

*2.2.2 Justification.* A User can have multiple different notes, all of which have a module and course assigned to them (NM6). A user can be either a student or a lecturer, and will have different functions in the system (AM7). The user can make suggestions to edit notes (NM3) and then the owner can edit the notes (NM4). Users can also rate notes (NM16). There will also be a leaderboard for students who are uploading or suggesting edits (GAM1 - 5). Both students and lecturers can upload notes (NM1) but only a lecturer can verify notes (NM5).

*2.2.3 Dynamic.*

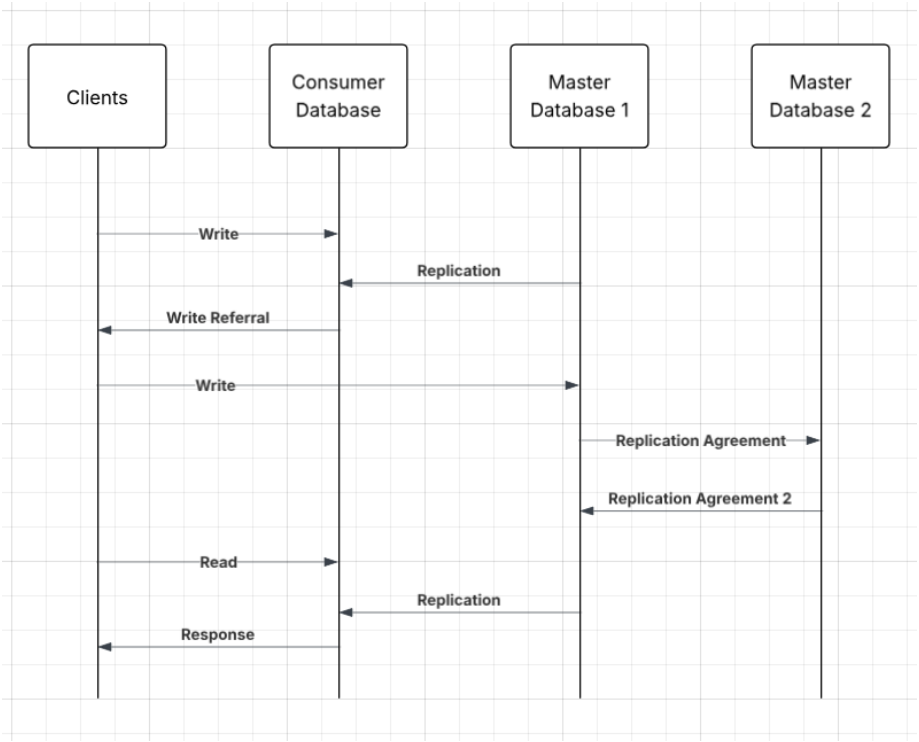


**Figure 3: Dynamic Architecture Diagram**

Table 10. Dynamic Element Catalogue

| Element | Purpose | Owner |
|---|---|---|
| Clients | Represents any user with an account who is trying to access the database. | N/A |
| Consumer Database | The database that will be pulled from to allow clients to see any notes/pages. | Jack |
| Master Database 1 | One of the databases that stores all notes/pages, can be written to by clients, and replicates changes to all other master databases. | Jack |
| Master Database 2 | Another database node for replication and redundancy. | Jack |

*2.2.4 Justification.* We have chosen to employ a Multi-Master Replication database, this is a database where multiple nodes of the database can accept write requests. Once any of these 'master' nodes receives a write request it will accept and apply the changes, then will propagate these changes to all other master nodes. This also means that every master node acts as a replica to all others. The reason this will be used is that it is very likely multiple notes will be uploaded at the same time and this could also help increase the scalability of the database (P1) in the future. The multi-master approach could cause a few issues of asynchronicity between users accessing the database, however this will likely not be too large a problem as multiple different versions will be uploaded, allowing them to view other notes if some have not yet been replicated. Also our database will allow for data to be read easily and quickly (P2). Deletion will also

be propagated during replication (P6, SEC5,AM2). The database will also use parameterised queries to prevent SQL injections (SEC7).

### 2.2.5 *Developmental.*

**Figure 4: Developmental Architecture Diagram**

Table 11. Developmental Element Catalogue

| Element | Purpose | Owner |
|---|---|---|
| Main Page | Start page to allow users to choose to login or sign up. | Meric |
| Login | Allow the user to log in. | Meric |
| Sign Up | Allow the user to create an account. | Meric |
| Register with School Email | University of Bath email needed to create an account. | Meric |
| User Interface | How the user will interact with the system. | Daniel |
| Home Page | Allow the user to navigate the website. | Daniel |
| Delete Account | Allow the user to delete their account. | Aidan |
| View Notes | Users can find notes sorted into courses and modules. | Aidan |
| Rate Notes | Users can leave ratings for notes. | Aidan |
| Suggest Edits | Users can suggest edits to the notes. | Daniel |
| View Leaderboard | Users can view the leaderboard. | Kourosh |
| Upload Notes | Users can upload notes. | John |
| Delete Their Notes | Users can delete notes they have uploaded. | John |
| Verify Notes | Lecturers can verify notes. | Victor |
| Verify Existing Notes | Lecturers can verify already uploaded notes. | Victor |
| Upload Verified Notes | Lecturers can upload verified notes. | Victor |

*2.2.6 Justification.* Each section of our website will give access to different features, to start a user must either create an account or login, requiring a University of Bath email (AM1). They are then redirected to the home page, which they interact with using the user interface. In this home page they can choose to delete their account (AM2,P6,SEC5) or continue. Both students and lecturers can read notes (NM17). Users can also download notes that they have viewed (ACC5), as well as this users are able to rate the notes that they read out of 5 (NM16) while viewing them. Users can also upload notes (NM1) , delete them (NM2),suggest edits to view ones (NM4) or edit their own (NM3,NM13). Lecturers can also verify notes that have been uploaded or upload their own verified notes (NM5). Users can also choose to view the leaderboard (GAM1-5)

*2.2.7 Deployment.*

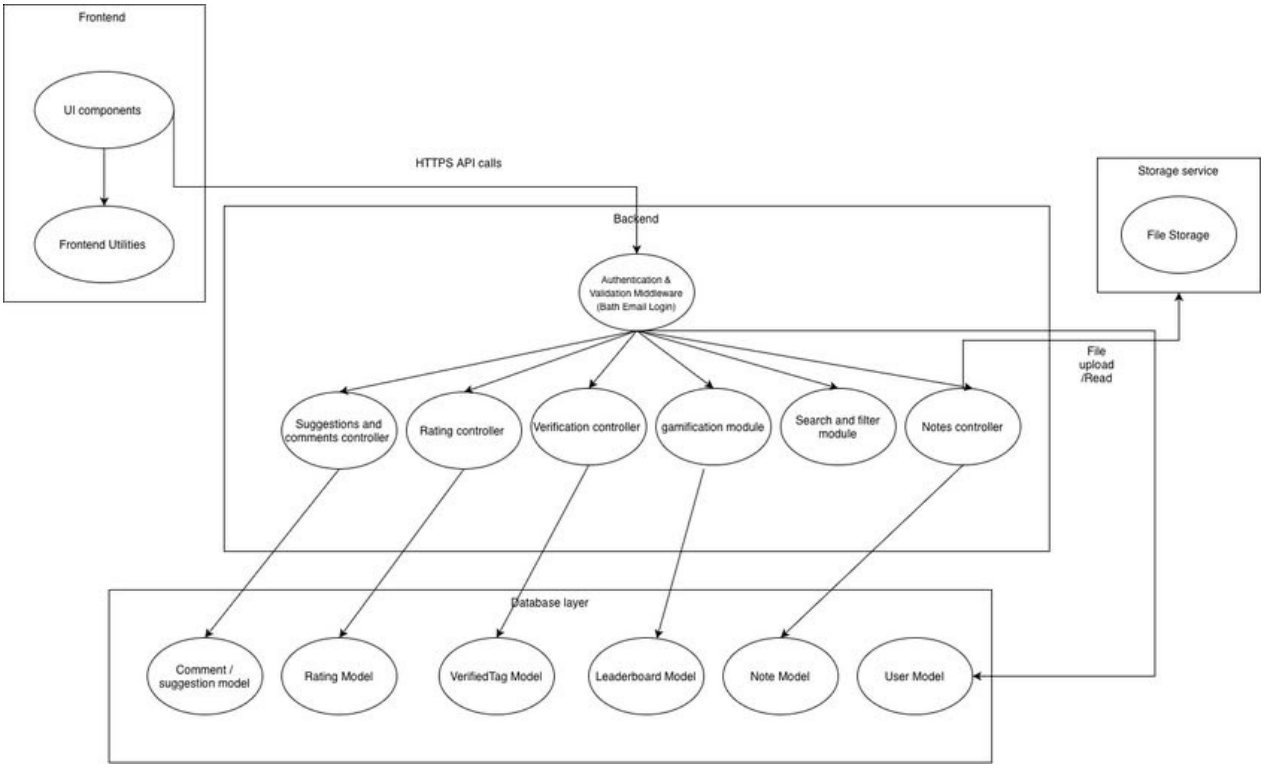


**Figure 5: Deployment Architecture Diagram**

Table 12. Deployment Element Catalogue

| Element | Purpose | Owner |
|---|---|---|
| UI Components | Renders note editor, dashboard, comments, ratings, filters, and module selection interfaces. | Meric |
| Frontend Utilities | Handles autosave (NM8), LaTeX rendering, input validation, and local UI state. | Meric |
| Auth & Validation Middleware | Handles Bath email-based login and enforces student/lecturer privileges (AM1, AM7). | Meric |
| Notes Controller | Implements creation, editing, deletion, and retrieval of notes (NM1–NM3). | Aidan |
| Suggestions & Comments Controller | Manages suggested edits and public comments (NM4). | Daniel |
| Rating Controller | Stores and updates 1–5 star note ratings (NM16). | Aidan |
| Verification Controller | Allows lecturers to verify / unverify notes (NM5). | Victor |
| Search & Filter Module | Filters notes by module, date, verification status, popularity, and reviews (NM11–NM12). | Caspar |
| Gamification Module | Implements levels, badges, ranking updates, and leaderboards (GAM1–GAM7). | Kourosh |
| User Model | Stores user identities, roles, and registered Bath email accounts (AM1). | Kourosh |
| Note Model | Stores note content, metadata, timestamps, and versioning (NM1–NM3, NM9). | John |
| Comment / Suggestion Model | Stores comments and proposed edits for collaborative improvements (NM4). | Daniel |
| Rating Model | Stores and aggregates numerical star ratings (NM16). | Aidan |
| VerifiedTag Model | Stores verification metadata from lecturers (NM5). | Victor |
| Leaderboard Model | Stores aggregated progress, XP, and ranking scores (GAM1–GAM7). | Caspar |
| File Storage Service | Stores uploaded PDFs, images, exports, and note attachments. | Jack |

*2.2.8 Justification.* The diagram shows NoteBuddy's module-level architecture. The frontend consists of reusable UI components and utilities for autosave, LaTeX rendering, and state management. It communicates with the backend only through secure HTTPS APIs. The backend contains the main application logic. Authentication and Validation manages login using Bath email accounts and enforces student/lecturer permissions (AM1, AM7). Notes, Suggestions/Comments, Ratings, and Verification Controllers handle core features such as note creation (NM1–NM3), collaborative edits and comments (NM4), rating updates (NM16), and lecturer verification (NM5). Search and Filter supports filtering by module, rating, verification, and date (NM11–NM12). Gamification features (GAM1–GAM7) are implemented in a dedicated module.

The database layer uses a replicated MongoDB cluster. The Storage Service handles PDFs, images, and exported files (ACC5). Authentication is fully managed by the backend without external SSO. This modular structure assigns each component a clear responsibility, improving maintainability and scalability.

## 3 Current System Status

For this iteration we planned to create a basic login and signup form for the website and create the database that will be used to store the login information and other data. We believe that this will be a good foundation to continue creating our project from and should allow for us to work well off of it.

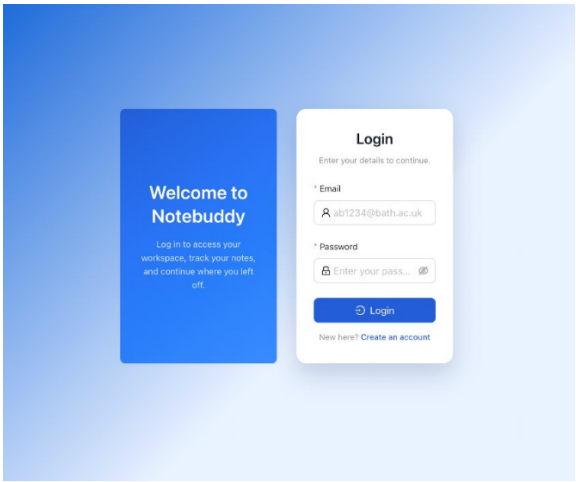


**Figure 6: Login Form**

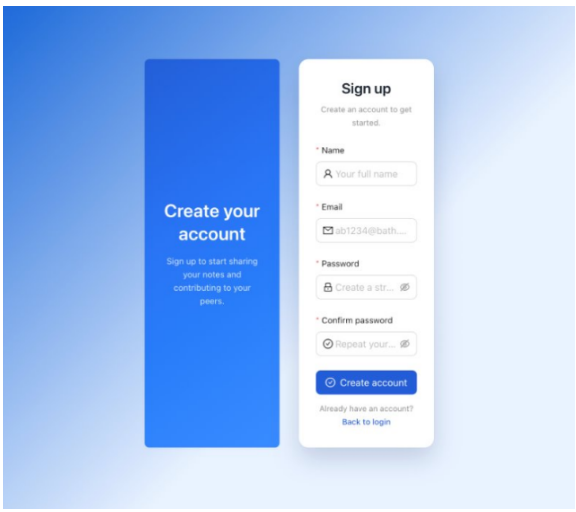


**Figure 7: Sign up Form**

We have taken different measures to ensure code quality. One of the measures we took is benefiting from ant-design libraries to have clean looking components throughout our user interface. This allowed us to create a more user-friendly environment with the colour palette and the components where students will feel motivated towards using the website.

```
import './App.css'
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Login from './Pages/Login';
import Register from './Pages/Register';

function App() {
  return(
    <Router>
      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path = "/" element={<Navigate to="/login"/>} />
      </Routes>
    </Router>
  )
}

export default App;
```

**Figure 8: Router Code**

We also used the react-router-dom library to import useNavigate in order to allow navigation between the login and register pages. Through creating the paths and routes within App.tsx we allowed the files within the Pages folder to be connected. Moreover, for the documentation of the code we used messaging, commenting, and having meetings to discuss the changes made to the code. For the external documentation, we are updating the README file regularly, making sure to communicate our updates with our stakeholders.

For the database, MongoDB has been installed within a codespace for the Git, and a created database has been connected to the code space so that the database can be both managed and edited easily, then tested within the code space. Node JS has also been installed, though it was only used for setup in this iteration, it may be very useful in future iterations.

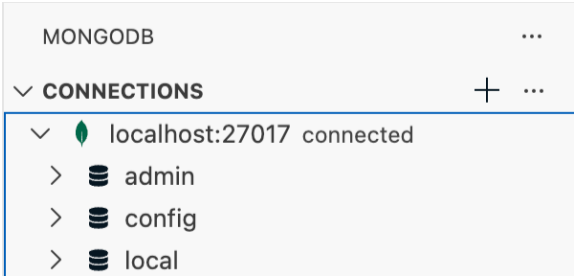


**Figure 9: Database**

Once the database was connected, a test user was added to the database to ensure that creating users would be possible, the first user was also created with read-only permissions, however an actual user would have write permissions to specific parts of the database (e.g. Comments on other peoples notes, pages containing their own notes)

**Figure 10: Database with user added**

Both the implementation of the database and adding the user were successful, and the required goal for this iteration was reached

During this iteration of NoteBuddy, we applied both Test-Driven Development (TDD) principles and Behaviour-Driven Development (BDD) to ensure the quality of our implemented login and signup features. TDD-inspired tests were used to verify the core functionality of our forms, such as validating required fields, checking password-matching, and ensuring that navigation between pages using "react-router-dom" behaved as expected. Alongside this, we carried out BDD-style testing to evaluate the system from the user's perspective. Using different scenarios, we tested how users would interact with the login and signup pages such as entering valid or invalid details and confirmed that the interface responded with the correct success or error messages. These combined testing approaches helped us ensure that NoteBuddy's foundational authentication UI is reliable, intuitive, and aligned with real user behaviour.

## 4 Project Management

### 4.1 Progress against initial plan

Since Deliverable 1A, we have made strong progress, Jira has been an effective platform for staying organised and consistently meeting deadlines within our Scrum workflow. Our initial objectives such as stakeholder interviews and literature reviews were fully completed, enabling us to define the functional and non-functional requirements for Deliverable 1B. These requirements were formalised in a structured MoSCoW-prioritised table (pages 9–13). Based on stakeholder research, we also developed SMART user stories aligned with key user goals. In addition, we advanced our architectural modelling by producing the Logical, Dynamic, Development, and Deployment views. We began development work by creating a GitHub repository for collaboration and version control, which was integrated with Jira. Despite this strong progress, a few aspects evolved differently than expected and required further attention.

### 4.2 Challenges Encountered

1. Evolving Requirements In Deliverable 1A, we expected to identify a straightforward set of user needs around viewing and sharing notes. However, following team discussions, and stakeholder as well as market research, our initial requirements for the website had evolved significantly. This added an important workload and expanded the functional scope we initially had in mind.

2. Benchmarking Comparable Systems Benchmarking Studocu and Studydrive revealed numerous problems such as forced uploads, poor subject filtering and lack of academic verification. This pushed us to rethink large parts of our initial design.

3. Uncertainty in Architectural Design Whilst designing the backend we encountered technical issues, like the handling of many simultaneous users. All of these questions required additional technical research to provide solutions we initially didn't have.

### 4.3 Corrective Actions Taken

1. Prioritisation Through MoSCoW As the number of desired features grew, we introduced MoSCoW prioritisation to control scope and ensure that core functionality remained the primary focus. We also held short consolidation meetings to sort out overlapping requirements and avoid unnecessary expansion. Less essential features were postponed to later stages, making the workload more manageable.

2. Corrective Actions Following Benchmarking Findings After discovering weaknesses in Studocu and Studydrive, we revised our design to avoid replicating their issues. We refined our assumptions from Deliverable 1A, ensuring our system addressed real gaps in our competition.

3. Technical Architecture Resolution To address our uncertainties around the architectural design, we focused research into distributed databases and replication strategies. This is how we decided to use a multi-master replication model to support simultaneous editing. We also created Architectural prototypes to validate decisions early on.

### 4.4 Departures from Targets

1. Greater Depth in Requirements Our requirements are now more detailed and filtered by importance. We now have functional and non-functional requirements, acceptance tests, scenarios, and prioritised features. This has enabled us to have a clear vision of our project goals.

2. Increased Technical Ambition The shift toward multi-master replication and more detailed architectural views are a clear improvement on our originally envisioned system. This also increases future development complexity.

### 4.5 Planned Changes Moving Forward

We will be prioritising the implementation of different features based on their importance in the requirements table. This is to manage the workload and ensure important features are prioritised. We also decided to start by designing a simpler single-master setup to ensure core functionality before scaling to multi-master designs.

### 4.6 Critique of Approaches

1. Requirements Establishment Interviews, surveys and MoSCoW helped prioritize features, but the sample size was small and may not represent broader users. Some requirements seemed based on assumption, and proposed solutions to stakeholder conflicts were not validated with users.

2. Benchmarking Comparable Systems Studocu and Studydrive provided useful feature comparisons, but the review focused mainly on functionality. Key areas such as performance, accessibility, and moderation were not analysed and the comparison lacked a structured framework.

3. Use Cases and User Stories Use cases were clear and included acceptance tests, but mostly covered ideal scenarios. Edge cases like inappropriate comments or simultaneous editing were missing.

4. Architecture and System Design Potential issues like data conflicts were not addressed, and the link between security/performance requirements and the architecture was sometimes unclear.

5. Current System Implementation Only login and signup screens are implemented. They are well designed, but the lack of core features prevents meaningful user feedback. Testing is minimal, and required accessibility features are not yet included.

6. Project Management and Workflow The team used Jira and GitHub effectively to organise tasks and track progress. However we still had inconsistencies between diagrams made by different members, showing that the modelling process was not fully aligned.

### 4.7 Re-evaluation of Risks

Several risks identified in Deliverable 1A have shifted as the project has progressed and the system design has become more detailed.

1.User Engagement, there still is risk of low participation, but its impact is reduced due to stronger incentives such as ratings, edit suggestions, leaderboards, and planned gamification features. These mechanisms provide clearer incentives for participation, although real engagement cannot be tested until there is a functional prototype.

2.Academic Integrity and Copyright, to mitigate concerns about students uploading copyrighted lecture slides or sharing inaccurate notes, features such as verification tags, rating system, and suggested edits were added. However the likelihood of problematic uploads remains moderate because content moderation systems are not implemented yet.

3.Accessibility, risk likelihood has decreased since clear accessibility requirements have been added (WCAG 2.1 compliance, screen reader support, contrast options). However, no accessibility features have been implemented in the current prototype, so the impact remains moderate until these are implemented and tested.

## References

[1] Studocu, 2025, Studocu Global - Study Notes for Students Worldwide,Retrieved from https://www.studocu.com/en-gb [Accessed 15th November 2025]

[2] Studydrive, 2025, Studydrive - Free university study notes and summaries, Retrieved from https://www.studydrive.net/ [Accessed 15th November 2025]

## A Appendix

### A.1 Survey

https://docs.google.com/forms/d/1dyzCdF06YiHkR9x1nbcvOXCK-hWlCvF2YdzTAaEwbqM/edit#responses